CMSC3621 - Data Structures and Algorithms
Lab Assignment 1 Report

Q1 - Nested Structs

Time and Space Complexity:
  Time: O(1) - Only fixed-size struct declarations and assignments; no loops.
  Space: O(1) - A constant number of struct variables allocated on the stack.

Source Code (Lab1_Q1.cpp):
  - Added missing main() function
  - Initializes struct3, struct2, and struct1
  - Links structs together via pointer (s2.mv3 = &s3)
  - Accesses mv6 through struct1 using: s1.mv2.mv3->mv6

Output:
  Value of mv6 accessed through struct1: Hello from mv6

```
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q1
Value of mv6 accessed through struct1: Hello from mv6
```

Q2 - Parentheses Validator (STL Stack)

Time and Space Complexity:
  Time: O(n) - Each character in the string is visited exactly once.
  Space: O(n) - Worst case (all opening parens), the stack holds n elements.

Fix Applied:
  Added #include <vector> - required because vector is used in main().

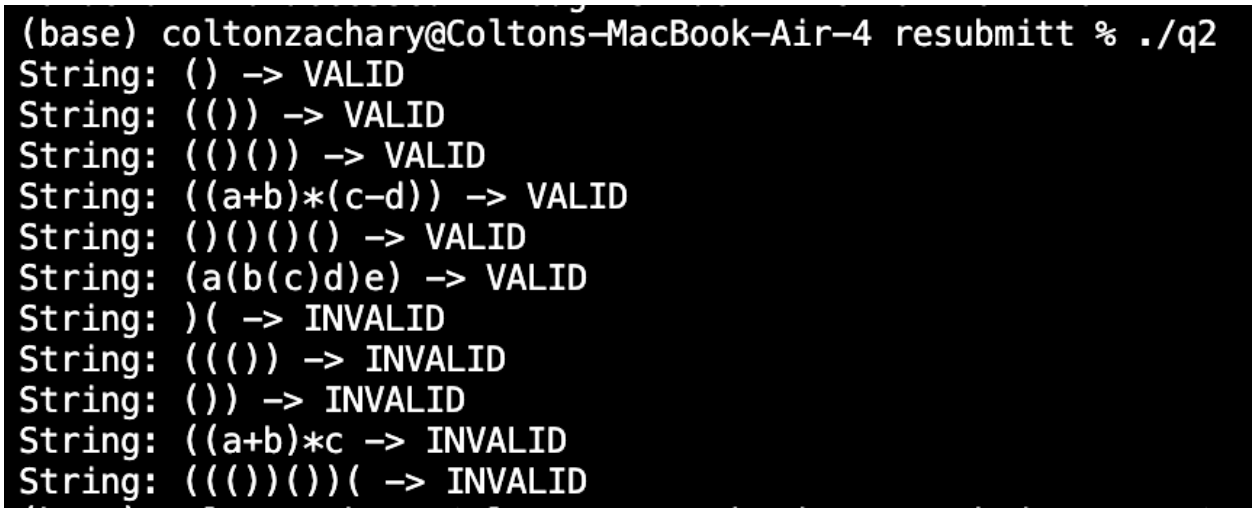
Algorithm:
  For each character: push '(' onto the stack; for ')', return false if the
  stack is empty, otherwise pop. Valid if the stack is empty after scanning.

Test 1 - Provided Examples:
  Input:
    ")(", "((()))", "()", "(a)((b)((c)(d)))"
  Output:
    String:)(-> INVALID
    String: ((()) -> INVALID
    String: () -> VALID
    String: (a)((b)((c)(d))) -> VALID

Test 2 - Full Test Suite:
  Input:

"()", "(())", "(()())", "((a+b)*(c-d))", "()()()()",
"(a(b(c)d)e)", ")(", "((()))", "())", "((a+b)*c", "((()))())("

Output:
String: () -> VALID
String: (()) -> VALID
String: (()()) -> VALID
String: ((a+b)*(c-d)) -> VALID
String: ()()()() -> VALID
String: (a(b(c)d)e) -> VALID
String:)(-> INVALID
String: ((())) -> INVALID
String: ()) -> INVALID
String: ((a+b)*c -> INVALID
String: ((()))())(-> INVALID



Q3 - Recursive Minimum (No Loops)

Time and Space Complexity:
Time: O(n) - One recursive call per element; n total calls.
Space: O(n) - The call stack grows to depth n before unwinding.

Algorithm:
Base case: return a[0] when size == 1.
Recursive case: find smallest in a[0..size-2], then compare with a[size-1].

Test 1:
Input: 13, 19, 12, 11, 15, 19, 23, 12, 13, 22, 18, 19, 14, 17, 23, 21
Output: smallest: 11

Test 2:
Input: 42, 7, 89, 56, 23, 95, 11, 68, 34, 77, 2, 61, 50, 14, 83, 29

Output: smallest: 2

Test 3:
Input: 5, 92, 38, 71, 16, 84, 47, 60, 13, 99, 28, 54, 3, 79, 41, 65
Output: smallest: 3

```
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % g++ -g Lab1_Q3.cpp -o q3
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % test 1
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q3
smallest: 11
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % g++ -g Lab1_Q3.cpp -o q3
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % test 2
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q3
smallest: 2
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % g++ -g Lab1_Q3.cpp -o q3
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % test 3
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q3
smallest: 3
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt %
```

Q4 - Pascal's Triangle Row (STL Queue)

Time and Space Complexity:
Time: O(n^2) - Building each of the n rows requires O(row_length) work.
Space: O(n) - The queue holds at most one full row at a time (n elements).

Algorithm:
Start with {1} in the queue. For each subsequent row, compute prefix sums
from the previous row's values, then append 1 at the end.

Tests:
Command: ./q4 3
Output: 1, 2, 1

Command: ./q4 4
Output: 1, 3, 3, 1

Command: ./q4 8
Output: 1, 7, 21, 35, 35, 21, 7, 1

```
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q4
Usage: ./q4 <row_number>
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q4 3
1, 2, 1
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q4 4
1, 3, 3, 1
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt % ./q4 8
1, 7, 21, 35, 35, 21, 7, 1
(base) coltonzachary@Coltons-MacBook-Air-4 resubmitt %
```